Locks, Sequencers, and Lock-delays

Let's explore how Chubby implements Locks and Sequencers.

Locks

Each chubby node can act as a reader-writer lock in one of the following two ways:

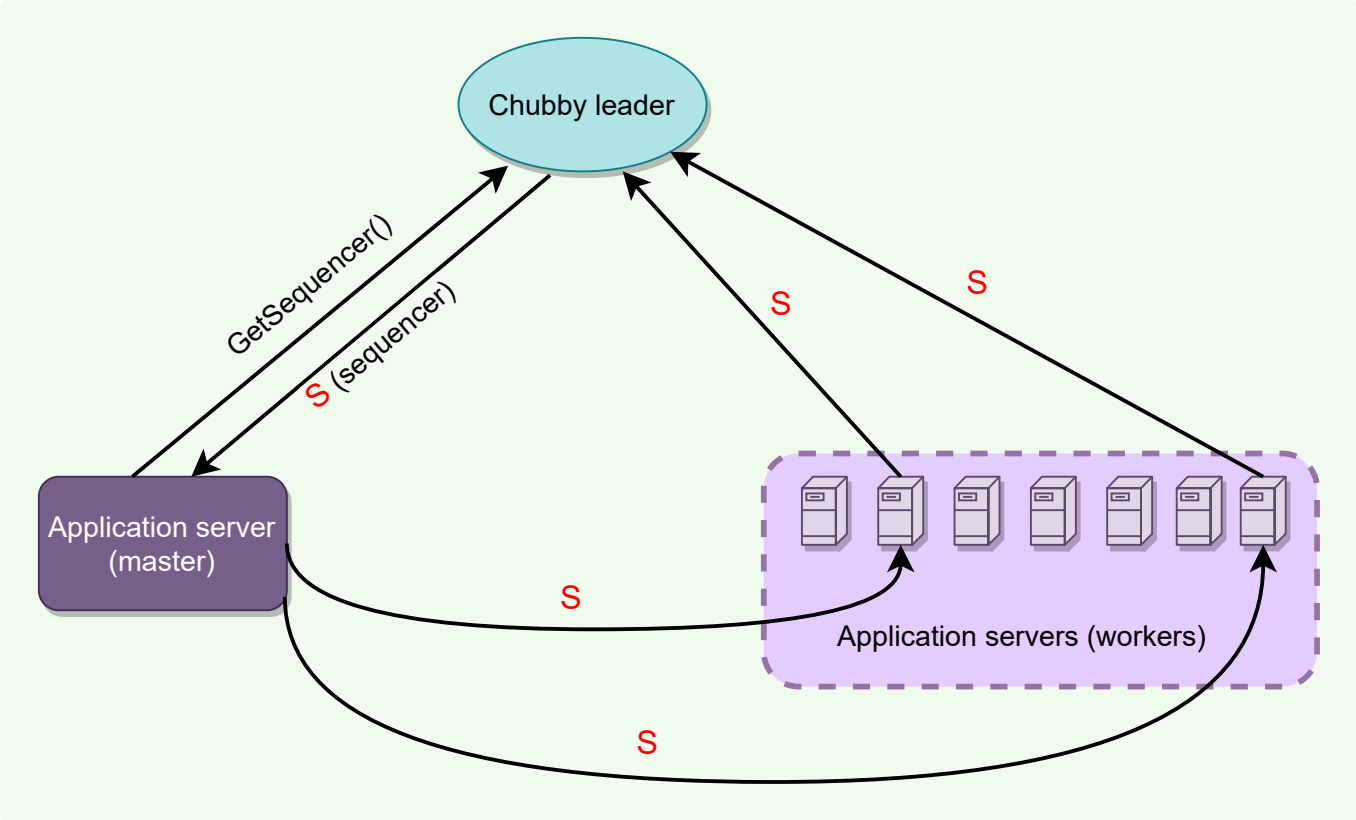
- **Exclusive:** One client may hold the lock in exclusive (write) mode.
- **Shared:** Any number of clients may hold the lock in shared (reader) mode.

Sequencer

With distributed systems, receiving messages out of order is a problem; Chubby uses sequence numbers to solve this problem. After acquiring a lock on a file, a client can immediately request a 'Sequencer,' which is an opaque byte string describing the state of the lock:

> **Sequencer** = **Name of the lock** + **Lock mode** (***exclusive or shared***) + **Lock generation number**

An application's master server can generate a sequencer and send it with any internal order to other servers. Application servers that receive orders from a primary can check with Chubby if the sequencer is still good and does not belong to a stale primary (to handle the 'Brain split' scenario).



Application master generating a sequencer and passing it to worker servers

Lock-delay

For file servers that do not support sequencers, Chubby provides a lock-delay period to protect against message delays and server restarts.

If a client releases a lock in the normal way, it is immediately available for other clients to claim, as one would expect. However, if a lock becomes free because the holder has failed or become inaccessible, the lock server will prevent other clients from claiming the lock for a period called the lock-delay.

- Clients may specify any lock-delay up to some bound, defaults to one minute. This limit prevents a faulty client from making a lock (and thus some resource) unavailable for an arbitrarily long time.
- While imperfect, the lock-delay protects unmodified servers and clients from everyday problems caused by message delays and restarts.